University of Asia Pacific
Department of Computer Science and Engineering
CSE 402: Operating System Lab

Lab Report: 01

Report Title: FCFS Implementation

Submitted by:

Name : Adika Sultana

Student ID: 23101162

Section : C2

Submitted to:

Atia Rahman Orthi
Lecturer ,
Department of CSE,UAP

Date of Submission: 03/08/2026

1. <u>Problem Statement:</u>

Implement FCFS CPU Scheduling for the following scenario where No of processes = 5 and Process informations are-

P id	AT	BT
P0	3	1
P1	5	3
P2	2	2
P3	1	2
P4	6	3

Calculate CT, TAT, WT, AVG TAT and AVG WT. Also represent the process execution sequence.

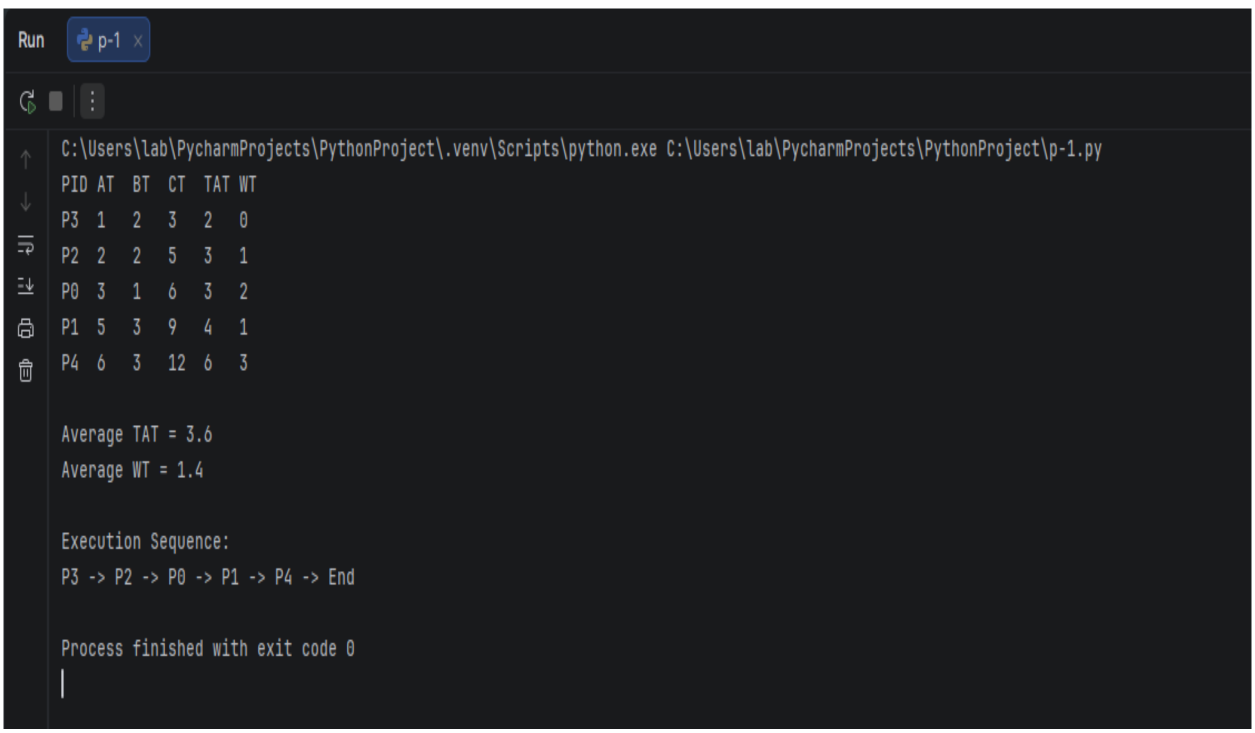
2. <u>Source Code:</u>

```
processes = [
    ["P0", 3, 1],
    ["P1", 5, 3],
    ["P2", 2, 2],
    ["P3", 1, 2],
    ["P4", 6, 3]
]
processes.sort(key=lambda x: x[1])

time = 0
total_tat = 0
total_wt = 0

print("PID\tAT\tBT\tCT\tTAT\tWT")

for p in processes:
    pid, at, bt = p


    if time < at:
        time = at

    ct = time + bt
    tat = ct - at
    wt = tat - bt

    total_tat += tat
    total_wt += wt
```

```
    print(f"{pid}\t{at}\t{bt}\t{ct}\t{tat}\t{wt}")

    time = ct

n = len(processes)

print("\nAverage TAT =", total_tat / n)
print("Average WT =", total_wt / n)

print("\nExecution Sequence:")
for p in processes:
    print(p[0], end=" -> ")
print("End")
```

GitHub: https://github.com/Adika2003/CSE-402-Operating-System-Lab

3. <u>Result:</u>



```
Run  p-1
C:\Users\lab\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\lab\PycharmProjects\PythonProject\p-1.py
PID AT  BT  CT  TAT WT
P3  1   2   3   2   0
P2  2   2   5   3   1
P0  3   1   6   3   2
P1  5   3   9   4   1
P4  6   3   12  6   3

Average TAT = 3.6
Average WT = 1.4

Execution Sequence:
P3 -> P2 -> P0 -> P1 -> P4 -> End

Process finished with exit code 0
```

4. Discussion:

The FCFS CPU scheduling algorithm was successfully implemented for the given five processes. FCFS follows a simple rule: the process that arrives first is executed first. Therefore, the processes were sorted according to their arrival times, producing the execution sequence P3 → P2 → P0 → P1 → P4**.** Since the first process, P3, arrived at time 1, the CPU remained idle from time 0 to 1. After that, each process executed continuously until completion because FCFS is a non-preemptive scheduling algorithm.

The program calculated Completion Time, Turnaround Time, and Waiting Time for every process. Turnaround Time was calculated using TAT = CT − AT**,** while Waiting Time was calculated using WT = TAT − BT**.** The condition used in the code also correctly handled CPU idle time when no process was available for execution.

According to the output, the average turnaround time was 3.6 time units**,** and the average waiting time was 1.4 time units**.** The calculated values and execution sequence matched the expected manual results. Therefore, the program correctly implemented FCFS scheduling and successfully produced all the required results.